

My Role

1a. After our first team meeting we divided the work among five: myself, Robin, Harish, Edward, and Zian. Harish took hardware setup and inter-component communications. Zian owned path planning. Robin owned the state machine. Edward and I shared computer vision—I delegated CV model training to Edward and kept the detection pipeline, calibration, and integration. That was the explicit split, and on paper it looked clean.

In practice my role drifted far beyond CV. I built the full detection-to-GPS-coordinate pipeline—a much bigger job than anyone anticipated. A single camera detection doesn’t give you a GPS position; you need focal length calibration, altitude compensation, and pitch-and-roll correction because the drone’s attitude shifts the image centre away from the nadir point. Individual GPS estimates had significant scatter, so I developed and benchmarked multiple averaging techniques—inverse-variance weighting, spatial clustering, Kalman filtering—each with different trade-offs. I tested all of these against the one real flight video we had (provided by Sid), which was literally all our real data. I used it for model evaluation, for retraining, and for deducing ground truth to benchmark my averaging methods against.

Alongside CV I took on the simulation framework as a “side thing.” It didn’t stay small. With hardware that wasn’t working, my laptop was the only place where forward progress was possible. The simulation grew into a thorough model of the full mission—GPS drift, camera shake, detection throttling, NFZ geofencing—and I ran it thousands of times, thinking through failure modes. I also built dashboards, monitoring tools, progressive test scripts for step-by-step confidence building, post-processing scripts for DJI footage, and field calibration tools. None of this was in my original brief. It happened because I kept seeing gaps between where the team was and where a working demo needed to be, and I couldn’t leave them unfilled. In hindsight, Tuckman (1965) would say we jumped from Forming into task execution without building the interpersonal trust that makes later conflict productive—our Forming was one meeting. When I look at the drift in my role, Lencioni’s (2002) framework explains part of it: building infrastructure alone was a way of feeling productive without having to slow down and bring others along. My instinct under pressure is that one person delivers. This project taught me that instinct is a failure mode dressed as a strength.

[APOLLO: 1a mentions “roles changed over the course of the project”—if there’s a specific week or event where your role visibly expanded (e.g. first failed flight day, Easter), name it here with a date.]

1b. I’m most proud of the simulation work and the CV-to-GPS estimation pipeline. Given extremely limited real data—one flight video from Sid—I managed to build a realistic way to estimate a casualty’s GPS position from camera detections, calibrate the camera properly in the lab (field of view, focal length at measured distances), and create an end-to-end simulation that could verify all twelve project requirements. On the first cancelled flight day, instead of going home, I calibrated the focal length, ran lens calibration, built field reference documents, and closed a stack of commits before dark. That day crystallised something: a missing drone must not become a missing report. The simulation framework is the only reason this project produced verifiable engineering output at all.

Edward trained a CV model as delegated, but we didn’t end up using it—we went with a standard YOLO person detection model, playing it safe. The reasons were sound: not enough flight days to collect realistic aerial training data, and we couldn’t validate a custom model in real conditions. [APOLLO: did you discuss this decision with Edward directly? Was he disappointed? One sentence on how that conversation went would strengthen this.]

1c. What I’d focus on differently is the gap between *capability* and *enablement*. I spent 3–4× more hours than expected on this project. Much of that time went into tools and infrastructure that only I could operate. In future work I want to spend less time building alone and more time making sure teammates can use, modify, and extend what I build without needing to ask me. The falsifiable signal I’ll watch for: can a teammate complete a task using my code within 48 hours of receiving it without coming back with clarifying questions? If they can’t, my documentation has failed, not their attention.

[APOLLO: add a specific future project or internship you’d apply this to—even one sentence grounds the lesson.]

My Team

2a. The team structure was discussed explicitly at the first meeting: Harish on hardware and component communications, Zian on path planning, Robin on the state machine, Edward and I on computer vision. We'd organised around tasks rather than around how those tasks would come together—efficient on paper, fragile in practice.

Edward and Zian weren't especially proactive. They did what they were assigned, and it looked like they did a fair amount of work, but they weren't self-directed—they didn't drive things forward on their own initiative. That's fair enough in a way; not everyone works the same. But it meant that when hardware failures killed our planned activities, those two didn't have a fallback. Robin and I were the two proactive members, working in close coordination especially on integrating my CV output with his state machine. The structure changed when hardware problems hit. We had two practice flight days, then Easter (with about one and a half extra days), then the demo. On the first two practice days the team had everything prepared—an in-depth simulation to test, progressive test scripts to run—but we were given faulty hardware. Things just didn't work, and we weren't sure what the issues were. All the other teams had working hardware from the start; ours wasn't functional until Easter, and the first two practice days effectively didn't count. There were also organisational problems: Wi-Fi was prioritised for teams actively flying, while others were supposed to tinker on the ground—but without connectivity we couldn't even configure our drone. There were IP address and naming conflicts. The hardware that wasn't working wasn't our fault—it was provided to us. [APOLLO: how diplomatically do you want to phrase the complaint letter reference? Draft a sentence here—e.g. “We raised these concerns formally in a letter to the course organiser” or softer.]

During Easter the team finally fixed the hardware problems and got confirmation that the state machine actually worked. With roughly one and a half functioning flight days, we produced a minimum viable product for the demo. It wasn't the standard we'd wanted, but we did the best we could given the constraints.

In Tuckman's (1965) terms, the external disruption pushed us backwards—from whatever tentative Norming we'd achieved into a demoralised re-Storming. As Graham noted in the Professional Practice sessions, “you can go from right to left—certainly your team could undevelop.” That regression is exactly what happened to us, and recovering from it consumed time we'd planned to spend testing.

[APOLLO: 2a asks “what changed over the course of the project?”—did anyone's role formally shift? Did Harish take on extra debugging once hardware failed? A concrete change to cite would strengthen this.]

2b. The element of team working that was most impactful was the close coordination between Robin and me during integration. After sorting out hardware issues, Robin needed detection output for his state machine. I provided captured images and GPS coordinates through a defined interface. Building that interface was expensive—attitude correction for pitch and roll, altitude compensation, dealing with significant single-estimate error. But once it existed, Robin could consume my CV output without touching my code, and we could develop in parallel. We discussed ideas as colleagues, not through formal handoff documents, and that directness was what made it work.

The most impactful *negative* element was the collective demotivation after the failed flight days. When hardware didn't work, the whole team felt it—myself included. I had loads of things I wanted to run and test. Instead of just being frustrated, I decided to push ahead in simulation. But that decision, while productive individually, also meant I was building alone rather than pulling the team forward. The honest assessment is that our most productive moments came when Robin and I were working *together* on integration, not when I was heads-down building infrastructure on my own. Lencioni (2002) describes how absence of trust leads teams to hide weaknesses and avoid asking for help. I think we partly fell into that pattern—retreating into individual work rather than confronting collective frustration openly.

[APOLLO: 2b is a prime spot for a disagreement or conflict episode. Did you and Robin ever disagree on something technical—e.g. interface design, confidence thresholds, approach to NFZ handling? Even a small one that was resolved quickly would satisfy the “manage conflict” gate for 62+.]

2c. Three things I'd change. First, I'd build in a weekly sync from week one—not a long meeting, just a brief check-in where we restate what each person is working on and flag blockers. The silent drift in our structure happened because we had no routine for restating priorities when the environment shifted.

When hardware failed, we didn't have a process for reallocating effort, and people without a fallback sat idle.

Second, I'd invest in interface contracts before parallel work begins. A short session in the first week to agree what each module expects as input and output would have saved significant integration pain later. Robin and I figured it out, but it cost time that upfront design would have prevented.

Third, I'd address the proactivity gap earlier and more directly. Edward and Zian did their assigned work but didn't self-direct. In future I'd try to understand what's blocking someone from taking initiative—unclear expectations, lack of confidence, or something else—rather than working around it by doing more myself. As Graham's Professional Practice lectures emphasised, "a group of individuals is not a team." I think we were closer to the former than the latter for most of the project, and that's partly on me for not pushing harder to change it.

[APOLLO: is there a specific conversation or moment with Edward or Zian you could cite? Even "I asked Edward in week X whether he needed help and he said no"—one sentence of real dialogue would anchor the proactivity observation.]

My Impact

3a. The CV pipeline and the simulation framework were my two largest contributions to the team's output. I'll explain the mechanism behind each, because "I built a lot" isn't a useful claim—what matters is how the work drove the project forward and where it fell short.

The CV pipeline started with my formal responsibility: detection. But detection alone is nearly useless to a SAR drone. You need to convert a pixel bounding box into a GPS coordinate the flight controller can act on. That conversion is deceptively expensive. It requires calibrated focal length, real-time altitude, and attitude correction for pitch and roll—because the drone is almost never perfectly level, and even a few degrees of tilt shift the image centre away from the nadir point. I did thorough camera calibration in the lab: field of view at measured distances, focal length measurements, lens distortion mapping via checkerboard. Single detection estimates had significant scatter, so I developed and tested multiple averaging approaches—inverse-variance weighting, spatial clustering, Kalman filtering—each with different trade-offs in latency versus accuracy. I benchmarked all of them against ground truth I deduced from the one real flight video Sid provided. That single video was literally all our real aerial data, and I squeezed everything I could out of it: model evaluation, retraining data, calibration validation, and GPS accuracy benchmarking.

The pipeline's interface was deliberately clean. Robin's state machine could call the detection function and get back a found/not-found flag, pixel coordinates, and a confidence score without knowing anything about the internals. When Robin needed GPS estimates for his flight logic, the interface extended naturally. That separation meant we could develop in parallel—he didn't wait for me, and I didn't break his code when I changed averaging methods or swapped models.

Edward trained a CV model as I'd delegated. We didn't end up using it. We went with a standard YOLO person detection model instead, and the reasoning was sound: we didn't have enough flight days to collect realistic aerial training data, and we couldn't run proper experiments to validate a custom model in real outdoor conditions. We did have trained models for detecting specific items of interest, but the YOLO person detector was the safer choice given our limited real-world testing. Playing it safe with a proven model rather than gambling on an untested custom one felt like the right engineering call. [APOLLO: how did you communicate this decision to Edward? Was he okay with it, disappointed, or did he agree it made sense?]

The simulation came out of frustration turned productive. After the first two practice days produced nothing—faulty hardware, no GPS to test against, the whole team demoralised—I had a choice: wait for the next flight window or find a way to make progress without a drone in the air. I chose simulation. What started as a quick sanity check grew into a thorough model of the full mission: GPS drift, camera shake, detection throttling at realistic inference speeds, the complete state machine, and all twelve project requirements verifiable end-to-end. I ran it thousands of times, watching closely, thinking through what could go wrong. Instead of holding failure modes in my head, I had something concrete to test against. Robin suggested the repulsive force concept for NFZ avoidance; I thought it was a great idea, got into it, and spent several days experimenting with edge cases—what happens when the target sits right on the

no-fly zone boundary, how the repulsive field should decay with distance, what override behaviour the pilot needs. The geofence and safety layer that came out of this was one of the more satisfying pieces of engineering in the project.

I also built dashboards and monitoring tools—a passive watch dashboard for detection during manual flight, a diagnostics dashboard showing all drone subsystems at once with live model switching—all running headlessly on the Pi through a browser. I wrote progressive test scripts forming a safety-critical test ladder: hardware checks, passive observation, waypoint tests, then full autonomous runs, each designed to build confidence before adding risk. The risk assessment framework from Graham’s Professional Practice lectures (the five-step HSE process) shaped how I structured that progression: identify hazards, determine who might be harmed, evaluate risks, record findings, review.

Everything came together on the actual flight day without prior real-hardware testing of the full integrated pipeline. That worked because the simulation had been thorough—but it was also somewhat lucky. In risk management terms, we accepted a higher residual risk than I was comfortable with, because we’d simply run out of time. The fact that the demo went well says something about the quality of the preparatory work, but I’d have been much more confident if we’d had time to run the progressive test ladder step by step.

3b. The honest gap is that I built things only I could maintain. Nobody else on the team edited the simulation code or the CV pipeline. If I’d been unavailable, the team’s most complex modules would have been unreadable to anyone else. I built things that worked, but I didn’t build things others could extend. That’s a prototype, not engineering.

In future projects I’d refocus on three things. First, every tool I build needs at least one named teammate who can use it independently—if I can’t identify that person before I start coding, the design needs rethinking. Second, I’d pair on code reviews for any file over 500 lines rather than refactoring solo. Third, I’d invest earlier in documentation that isn’t just technically complete but genuinely usable by someone who didn’t write the code. The test: can a teammate modify the module within 48 hours without asking me how? If they can’t, my documentation has failed, not their attention.

I’d also spend less time retreating into technical work when things got frustrating. When hardware failed and the team was demoralised, my instinct was to go heads-down on my laptop and build things. That was productive in output terms, but it wasn’t leadership. Graham’s observation that “the forced detachment from the technical work which might have been the reason why you entered engineering can be too much to bear” hit close to home. In future I want to balance individual delivery with enabling others—even when enabling others feels slower than doing it myself. [APOLLO: any specific future project, internship, or career direction you want to reference as where you’ll apply this?]

My Support

4a. Robin was the teammate who helped me work most effectively, and I can point to the specific mechanism. After the Easter hardware fix, Robin needed my detection output for his state machine. That meant we had to agree on an interface: what data my pipeline would provide, in what format, at what rate. Working through that contract forced me to make my code cleaner and my GPS estimation more robust than it would have been if I’d only been building for myself. Robin was proactive—he pushed back when things weren’t clear and held me to delivering something he could actually use on a specific day. That pressure made my work better. In Lencioni’s (2002) terms, we’d built enough trust at the base of the pyramid that disagreement was productive rather than political: I could say “this estimate is noisy, give me a day” and he’d say “fine, but I need it by Thursday,” and that was a conversation, not a conflict. Robin also suggested the repulsive force concept for NFZ avoidance. I could have treated that as an intrusion into my module, but I thought it was a genuinely good idea and ran with it. The fact that we could exchange ideas as equals—rather than defending territorial boundaries—was what made the integration period work.

[APOLLO: what specifically did Harish do that helped you or unblocked the team? Even one concrete example—e.g. wiring the Cube to the Pi, sorting a connectivity issue, anything.]

[APOLLO: did Edward or Zian do anything that directly helped your work? A dataset contribution, a test, feedback on your code, anything at all?]

[APOLLO: Sid provided the one DJI flight video with GPS telemetry. How did that happen—did you

ask for it, or did he offer? Any context on the interaction?]

[APOLLO: did you receive any feedback from teammates during the project—positive or critical—that changed how you worked? Even informal comments count.]

4b. What I did for the team splits into technical enablement and communication, and I want to be honest about which actually landed and which didn't.

The technical contributions that *did* land: the CV pipeline interface let Robin's state machine consume detection output without touching my code. Once the interface existed, Robin could develop his flight logic in parallel. I also built the simulation as something the whole team could reference—I communicated the work to the team, and they were happy with it. Robin used it as a reference point for his integration work. The safety-critical layer—zone repulsive fields, geofencing, kill switch handling—was designed so that any team member operating the system would be protected from dangerous edge cases without needing to understand the internals.

The technical contributions that *didn't* land as intended: the progressive test scripts. I designed a full safety-critical test ladder—bench checks through passive flight observation to full autonomous mission—each step building confidence before adding risk. Due to limited flight time, the suite was never fully executed on real hardware. The scripts were well-designed in isolation, but I didn't tie them to a specific team activity or flight day plan tightly enough for them to get used. In Lencioni's framework, accountability only works when commitments are specific—vague intentions to “make tools available” don't drive behaviour the way “we're running test 2 at 10am tomorrow” does. That's a lesson I'll carry forward: tooling without a named user and a deadline is tooling that gathers dust.

The dashboards sat somewhere in between. The passive watch dashboard and the diagnostics dashboard—both headless, browser-based, designed to run on the Pi—were ready and functional. [APOLLO: did the team actually use these dashboards on flight days, or only you? Be honest.]

On the communication side, I drafted a complaint letter about the hardware situation. [APOLLO: how did the team react to the complaint letter? Did anyone help edit it? How was the final version agreed before it went out with all five names?] The letter went to Steve. That was an exercise in what Graham's risk management lectures call the moral pillar of obligation: when safety-critical equipment doesn't work, staying silent isn't professional. But I also learned that the line between honest escalation and complaining is thinner than I'd assumed. [APOLLO: did you get any feedback on the letter's tone—from teammates or from Steve?]

[APOLLO: did you do anything specific to help Edward or Zian with their work? Explain a concept, review their code, provide data, pair on a task? Even small things count.]

[APOLLO: did you give feedback to any teammate during the project? Positive or constructive? How was it received?]

The broader pattern I see is that my strongest support contributions were the ones with a named recipient. The CV interface worked because Robin needed it on a specific day. The simulation was useful because it solved a concrete problem the whole team could see. The test scripts and dashboards were technically sound but lacked that same pull—nobody was waiting for them. In future I'd make sure every piece of infrastructure I build has someone who's asked for it, not just someone who might use it.

[APOLLO: closing thought—what are you doing next (internship, career direction, Ukraine focus), and how has this project shaped your thinking? One or two sentences to close.]